上海大学 2015～2016 学年度秋季学期试卷 (A 卷)

成绩	

课程名：面向对象程序设计 课程号：08305121 学分： 5

应试人声明：

我保证遵守《上海大学学生手册》中的《上海大学考场规则》，如有考试违纪、作弊行为，愿意接受《上海大学学生考试违纪、作弊行为界定及处分规定》的纪律处分。

应试人 _____ 应试人学号 _____ 应试人所在院系 _____

题号	一(20)	二(20)	三(20)	四(40)
得分				

得分	

一、判断题（每小题 2 分，共 20 分）

1. 类的构造函数的函数名与类名相同，可以重载构造函数。 **(√)**
2. 类的析构函数可以被重载。 **(×)**
3. 重载运算符函数不能改变运算符的操作数个数、优先级和结合方向。 **(√)**
4. 引用在声明时必须对其初始化，以绑定某个已经存在的变量（或对象），在该引用的生命期内，该绑定不能被更改。 **(√)**
5. 指针变量在定义时必须对其初始化，以锁定某个已经存在的目标变量（或对象），在该指针变量的生命期内，该指向不能被更改。 **(×)**
6. 类的非静态成员函数均有一个隐含的形式参数 this 指针常量，用于指向调用该函数的对象。函数体中不能改变该指针常量的指向（即锁定调用该函数的对象）。 **(√)**
7. 派生类继承了基类的所有数据成员，并且在派生类的成员函数中都能直接访问基类的访问属性为 private 的成员。 **(×)**
8. 构造派生类对象时，先调用基类的构造函数，后执行派生类的构造函数。析构派生类对象时，先调用基类的析构函数，后执行派生类的析构函数。 **(×)**
9. 含纯虚函数的类称为抽象类，不能创建抽象类的对象，不能定义抽象类的指针变量，不能声明抽象类的引用。 **(×)**
10. 引用返回的函数可以作左值，也避免了函数值返回时创建与返回类型相同的临时无名对象。 **(√)**

得分 | | 二、填空题（每空 2 分，共 20 分）如下设计了一个字符串类 String ，请根据运行结果，完成程序。

```
#include    <iostream>
#include    <cstring>
#include    <string>
using    namespace    ①  std   ;

class    String
{
public    :
      String(     const     char    *str="")
      {
      size = strlen(          ②  str    );
      x = size>0 ?          new   char  [size] : NULL;
      if   (x==NULL) size = 0;
      for   ( int    i=0; i<size; i++)
            x[i] = str[i];
}
String(     const    String &s) : x(          ③  NULL   )
{
      * this    = s;                                    //    直接利用深赋值运算符函数
}
virtual       ~String()
{
      if   (x!=NULL)      delete      ④  []    x;
      size = 0;
}
String &        operator     =( const    String &s)
{
      if   ( this    ==    ⑤  &s    )       return     *  this   ;
      if   (x!=NULL)      delete     [] x;
      size = s.size;
      x =     new   char  [size];
      if   (x==NULL) size = 0;
      for   (int i=0; i<size; i++)
            x[i] = s.x[i];

      return      ⑥    *this        ;
}

⑦   char   &       operator     [](   int    index)       {     return     x[index]; }

friend     ostream &       operator    <<(ostream &out,          const    String &s)
{
```

```
        for   ( int    i=0; i<s.size; i++)
              out << s.x[i];
        return      ⑧  out    ;
}
⑨    friend        istream &          operator     >>(istream &in, String &s)
{
        string str;
        in >> str;                                          //   利用 C++ 字符串
        s = String(str.c_str());                            //   利用深赋值运算符
        return     in;
}
friend      ⑩    int      Compare(     const    String &s1,          const    String &s2)
{
      int    i;
      for   (i=0; i<s1.size && i<s2.size && s1.x[i]==s2.x[i]; i++)
            ;
      if   (i<s1.size && i<s2.size)
            return     s1.x[i] > s2.x[i] ? 1 : -1;
      else     if   (i<s1.size && i==s2.size)
            return     1;
      else     if   (i==s1.size && i<s2.size)
            return     -1;
      else
            return     0;
}
friend bool operator              <( const    String &s1,          const    String &s2)
{     return     Compare(s1, s2) < 0; }
friend bool operator              <=( const    String &s1,          const    String &s2)
{     return     Compare(s1, s2) <= 0;}
      friend bool operator              >( const    String &s1,          const    String &s2)
      {     return     Compare(s1, s2) > 0; }
      friend bool operator              >=( const    String &s1,          const    String &s2)
      {     return     Compare(s1, s2) >= 0;}
      friend bool operator              ==( const    String &s1,          const    String &s2)
      {     return     Compare(s1, s2) == 0;}
      friend bool operator              !=(   const    String &s1,          const    String &s2)
      {     return     Compare(s1, s2) != 0;}
protected      :
      char    *x;
      int     size;
};
```

```
void    display(       const    String &s1,          const    String &s2)
{
      char    *str[] = {"            小于", "     等于", "     大于"};
      cout << "\"" << s1 << "\" " << str[1+Compare(s1, s2)]
            << " \"" << s2 << "\"\t" << endl;
}
int     main()
{     String s1("Hello world!"), s2(s1);
      display(s1, s2);
      s2[0] = 'h';
      display(s1, s2);
      s2 = "Hello world ";
      display(s1, s2);
      s2 = "Hello world";
      display(s1, s2);
      s1 = "";              s2 = "";
      display(s1, s2);
      return      0;
}
```

运行结果

```
"Hello world!"          等于  "Hello world!"
"Hello world!"          小于  "hello world!"
"Hello world!"          大于  "Hello world "
"Hello world!"          大于  "Hello world"
""     等于  ""
```

得分	

三、阅读程序写出运行结果（每行　1 分，共 20 分）

3.1（10 分）本题所涉及的 Time 类，相关头文件和源程序文件如下。

// MyTime.h　　头文件

```
#ifndef      MYTIME_H
#define      MYTIME_H
#include      <iostream>
#include      <iomanip>
using     namespace    std;

class     Time
{
public     :
      Time(  int    hour=0,       int    minute=0,       int    second=0);
      Time &     operator     ++();
      Time    operator     ++(  int   );
      friend      Time     operator     +(  const    Time &t,       int    n);
      friend      ostream &       operator     <<(ostream &out,          const    Time &t);
      friend      istream &       operator     >>(istream &in, Time &t);
protected      :
      int    h, m, s;
};

#endif
```

```
// MyTime.cpp                                                    源程序文件
#include     "MyTime.h"
Time::Time(int hour, int minute, int second)
                        : h(hour), m(minute), s(second)                    //   构造函数
{
}
Time & Time::       operator     ++()
{
      s++;
      if   (s==60){        s = 0; m++;        }
      if   (m==60){        m = 0; h++;        }
      if   (h==24)         h = 0;
      return     *  this    ;
}
Time Time::       operator     ++( int    )
{
      Time temp(*      this    );
      ++(*   this    );
      return     temp;
}
Time    operator     +( const    Time &t,        int    n)
{
      Time result(t);
      int    x = (t.h*60 + t.m)*60 + t.s + n;
      while   (x<0)
            x += 24*60*60;
      x %= 24*60*60;
      result.s = x % 60;
      result.m = x/60 % 60;
      result.h = x/3600;
      return     result;
}
ostream &       operator     <<(ostream &out,          const    Time &t)
{
      out << setfill('0') << setw(2) << t.h << ':'
            << setw(2)<<t.m << ':' << setw(2)<<t.s << setfill(' ');
      return     out;
}
istream &       operator     >>(istream &in, Time &t)
{
      char   str[200];
      in.getline(str, 200, ':');
      t.h = atoi(str);
      in.getline(str, 200, ':');
      t.m = atoi(str);
```

```
        in.getline(str, 200);
        t.s = atoi(str);
        return      in;
}

// main.cpp                源程序文件（测试程序）
int     main()
{
        Time t0(23,59,50), t;
        t=t0; cout << ++t << endl;
        t=t0; ++t;    cout << t << endl;
        t=t0; ++++t; cout << t << endl;
        t=t0; cout << t++ << endl;
        t=t0; t++;              cout << t << endl;
        t=t0; t++++; cout << t << endl;
        t=t0; t=t+(-3600); cout << t << endl;
        cout << "       请输入时间 (hh:mm:ss) : ";
        cin >> t;
        cout << t << endl;
        cout << ++t << endl;
        cout << t + (10*60+20)*60+30 << endl;
        return      0;
}
```

运行结果 (3.1)

```
23:59:51
23:59:51
23:59:52
23:59:50
23:59:51
23:59:51
22:59:50
请输入时间 :23:59:59
23:59:59
00:00:00
10:20:30
```

3.2（10 分）以下 4 小题所涉及的 Test1 类，相关头文件和源程序文件如下。

```
// test03.h                                                    头文件
#ifndef      TEST03_H
#define      TEST03_H
#include      <iostream>
using     namespace   std;
class     Test1
{
public     :
        Test1(    int     a=0);
        Test1(    const     Test1 &t);
        virtual        ~Test1();
        Test1 &       operator      =( const     Test1 &t);
        static        int     Num();
        static        int     Sum();
        friend      ostream &         operator     <<(ostream &out,          const     Test1 &t);
        friend      istream &         operator     >>(istream &in, Test1 &t);
protected      :
        static        int     num, sum;
```

```
    int    x;
};
void    Show();                                          //    普通的  C++ 函数声明
#endif

// Test03.cpp                                                源程序文件
#include     "Test03.h"

int    Test1::num=0, Test1::sum=0;                        //  静态数据成员定义及初始化

Test1::Test1(        int    a):x(a)                       //    构造函数
{
    num++; sum+=x;
}
Test1::Test1(        const    Test1 &t):x(t.x)                    //    拷贝构造函数
{
    num++; sum+=x;
}
Test1::~Test1()
{
    num--; sum-=x;
}
Test1 & Test1::          operator     =( const    Test1 &t)       //    赋值运算符函数
{
    sum += t.x - x;
    x = t.x;
    return     *  this    ;
}

int    Test1::Num() {          return     num;}

int    Test1::Sum() {          return     sum;}

ostream &       operator     <<(ostream &out,          const    Test1 &t)
{
    out << t.x;
    return     out;
}
istream &       operator     >>(istream &in, Test1 &t)
{
    int temp;
    in >> temp;
    Test1::sum += temp - t.x;
    t.x = temp;
    return     in;
}
```

```
void    Show()                                          //    普通的 C++ 函数
{
      cout       << "Num = " << Test1::Num()
                 << ",\tSum = " << Test1::Sum() << endl;
}

// 3.2.1   测试程序之一
#include      "Test03.h"
int     main()
{
      Show();
      return      0;
}
```

运行结果 (3.2.1)

Num = 0 , Sum = 0

```
// 3.2.2   测试程序之二
#include      "Test03.h"
Test1 x(100);                                 //    创建一个全局对象
void    f(Test1 t)
{
      Show();
}
int     main()
{
      Show();
      f(x);
      Show();
      return      0;
}
```

运行结果 (3.2.2)

Num = 1 , Sum = 100

Num = 2 , Sum = 200

Num = 1 , Sum = 100

```
// 3.2.3   测试程序之三
#include      "Test03.h"
void    f(Test1 &t)
{
      Show();
}
int     main()
{
      Test1 x(100);              //    创建一个自动对象
      Show();
      f(x);
      Show();
      return      0;
}
```

运行结果 (3.2.3)

Num = 1 , Sum = 100

Num = 1 , Sum = 100

Num = 1 , Sum = 100

```
// 3.2.4  测试程序之四
#include     "Test03.h"
int    main()
{
       Test1 x(10),y(20),a[2]={x,y}
       Show();
       Test1 *p =        new Test1;
       *p = 30;
       Show();
       delete     p;
       Show();
       return     0;
}
```

运行结果 (3.2.4)

Num = 4 , Sum = 60

Num = 5 , Sum = 90

Num = 4 , Sum = 60

得分	

四、（40 分）设计复数类。 要求运行时得到指定的输出结果。

① 实现如下测试程序中用到的 9 个函数（每个函数 3 分。无须定义拷贝构造函数、析构函数及赋值运算符函数）；

② 自选 3 个运算符，并实现运算符函数重载（每个函数 3 分。注意复数不能比较大小）；

③ 数据成员、类设计的其他部分（ 4 分）。

【注意：数学函数 double atan2(double y, double x); 当 x≠ 0时返回 y/x 的反正切值，当 x=0 时返回 π/2 或- π/2 （正负号与 y 同号）】。

```
#include     "Complex.h"

int    main()
{
       Complex x(3,4), y(x), z;                                  //   创建对象
       cout << x << ", " << y << ", " << z << endl;              //   输出复数
       cout << x.Abs() << '\t';                                  //   计算复数的模长
       z.Real() = z.Imag() = 1;                                  //   设置复数的实部、虚部
       cout << z.Angle()*180/M_PI << endl;                       //   计算复数的角度
       cout << x+z << '\t';                                      //   复数的算术运算 ：+、*，迭代赋值 *=
       cout << x*z << '\t';
       cout << (x*=x) << endl;
       return     0;
}
```

运行结果 (4.1)

(3, 4), (3, 4), (0, 0)

5 45

(4, 5) (-1, 7) (-7, 24)

```
// Complex.h
#include <iostream>
#include <cmath>
using namespace std;

class Complex
{
public:
	Complex(double real=0, double imag=0):re(real),im(imag){}

	double & Real(){ return re; }
	double & Imag(){ return im; }
	double Angle() const { return atan2(im, re); }
	double Abs() const { return sqrt(re*re + im*im); }

	friend Complex operator+(const Complex &c1, const Complex &c2)
	{
		Complex result(c1);
		result.re += c2.re;
		result.im += c2.im;
		return result;
	}

	friend Complex operator*(const Complex &c1, const Complex &c2)
	{
		Complex result;
		result.re = c1.re*c2.re - c1.im*c2.im;
		result.im = c1.re*c2.im + c1.im*c2.re;
		return result;
	}

	Complex & operator*=(const Complex &c)
	{
		double x = re*c.re - im*c.im;
		im = re*c.im + im*c.re;
		re = x;
```

```
        return *this;
    }

    friend ostream & operator<<(ostream &out, const Complex &c)
    {
        out << '(' << c.re << ", " << c.im << ')';
        return out;
    }

protected:
    double re, im;
};
```